Assignment 4

alfar24@ru.is, hlynurh24@ru.is



T-519-STOR: Theory of Computation

Problem 1: the complexity class NP (30 points)

1. (15 points) Assume $A \in$ NP and $B \in$ P. Show that $A \setminus B \in$ NP

Solution:
Let M_A be a nondeterministic Turing machine that decides A in polynomial time, Let M_B be a deterministic Turing machine that decides B in polynomial time, we construct a nondeterministic Turing machine M that decides $A \setminus B$ in polynomial time:

Let $M =$ "On input $\langle w \rangle$:

1. Run M_A on input $\langle w \rangle$
2. If M_A rejects, reject,otherwise run M_B on w
3. If M_B accepts, reject, if M_B rejects, accept"

Correctness:

As $A \setminus B$ means the set of strings that are in A but not in B, the decider M runs M_A to check if w is in A, if it is in A, we need to check if w is in B, as if it is in B, then it cannot be in$A \setminus B$, thus we accept only if w is in A but not in B, for time, since we know A is in NP, we know that M_A runs in polynomial time for a nondeterministic Turing machine, so running M_A takes polynomial time for M as M is nondeterministic, similarly M_B takes polynomial time for a deterministic Turing machine, so it takes polynomial time for a nondeterministic Turing machine, thus M runs in polynomial time and as it decides $A \setminus B$, $A \setminus B \in$ NP

2. (15 points) Assume that P = NP. Give a polynomial-time algorithm for finding a satisfying assignment for a boolean formula ϕ, if one exists.

Solution:
Given P = NP, there must exist a polynomial-time decider M_{SAT} for SAT. We construct a polynomial-time algorithm for finding a satisfying assignment for a boolean formula ϕ.
The algorithm will be done in pseudo code:

```
def Search Sat( φ ):
    Run  M_SAT  on  ⟨φ⟩
    if M_SAT rejects, output 0.
    Let vars be the set of all vars in φ
    Let φ' = φ
    Let t be a table of (var:bool)
    for i in vars:
        Set each instance of var i ∈ φ' to T.
        Run M_SAT on ⟨φ'⟩,
        if accept, add i:T to table t,
        if reject, then for all instances of i ∈ φ', assign i:F to table t.
    output φ'
```

Correctness:
Since the algorithm starts by running M_{SAT} on ϕ, we are guaranteed that ϕ is satisfactory. Then we just periodically pick variable values until we find a satisfying permutation.

Time complexity analysis
Given P = NP, a polynomial-time decider must exist for SAT. The rest of the algorithm runs in $O(n^{1+c} + n^2 + n^c)$, $n = |\phi|$, and c is some constant:
M_{SAT} runs in some polynomial-time $O(n^c)$.
We run M_{SAT} $n + 1$ times, once before the loop, and n times in the for loop, so $O(n^{c+1})$.
Setting each variable $i \in \phi$ is $O(n)$ but we do it $2n$ times, so $O(2n^2) = O(n^2)$.

Problem 2: Polynomial time reductions and NP-completeness

1. (20 points) Consider the language

$$B = \{\langle G, k, v\rangle \mid G \text{ is a graph with a } k\text{-clique containing } v\}$$

(a) Show that B is in NP.

(b) Give a polynomial-time reduction from CLIQUE to B.

(c) Is B NP-complete?

Solution:

a) Recall that for a graph, we denote it with $G = (V, E)$, where V is the set of vertices, let M_C be a non-deterministic Turing machine that decides CLIQUE that runs in polynomial time, we construct a nondeterministic Turing machine M that decide B.

Let $M =$ "On input $\langle G, k, v\rangle$:

1. For each $w \in V$, we check if $w = v$
2. If there exists a w, such that $w = v$, we run M_C on $\langle G, k\rangle$ and goto step 3, if no such w exists, we reject.
3. If M_C accepts, accepts, otherwise reject"

Correctness:

Based on the definition of B, it is almost the same as CLIQUE except the added condition of containing v, so, we start by checking if $v \in V$ and if so, we can just run the decider for CLIQUE, if it doesn't exist, then G does not contain v and we reject. CLIQUE is in NP, so there exists a non-deterministic Turing machine that decides CLIQUE in polynomial time, and as we have one for loop that goes through V, its also polynomial, so this machine runs in polynomial time and as such $B \in$ NP

b) To polynomially-time reduce CLIQUE to B, we must show that

$$\langle G, K\rangle \in CLIQUE \iff f(G, k) \in B$$

, so let us construct a Turing machine M' that reduces CLIQUE to B such, let us also recall the Turing machine M from (a)

Let M' = "On input $\langle G, k\rangle$:

1. Take any one $v \in V$
2. construct $f(x) = \langle G, K, v\rangle$
3. Run M on $f(x)$, accept if M accepts, reject if M rejects

To show this is correct, let us assume that $\langle G, k\rangle \in CLIQUE$, then we construct $f(x)$ such that we take any one $v \in V$ and add that to the input for M' so if CLIQUE accepts the input, then B accepts $f(x)$, for the other direction, if $f(x)$ is accepted by B, that means there is a k-clique, so CLIQUE accepts. this is polynomial, as it is polynomial to create the CLIQUE.

c) Since CLIQUE is polynomial-time reducible to B, B is NP-Hard, similarly we know that B is NP, so given these are the two conditions for NP-complete, we know that B is NP-complete

2. (20 points) For each boolean formula ϕ in conjuctive normal one (CNF) let max(ϕ) be the largest number of conjucts in ϕ that can be made true by some truth assignment to the variables occuring in ϕ. Consider the language

$$\text{M-SAT} = \{\langle \phi, k \rangle \mid \phi \text{ is a boolean formula in CNF and k} = \max(\phi)\}$$

Solution:

let us polynomialy-time reduce SAT to M-SAT, showing that

$$\phi \in \text{SAT} \iff f(\phi) \in \text{M-SAT}$$

Let m be the total number of conjuncts in ϕ, Let us construct a Turing machine that reduces SAT to M-SAT, and decides SAT based if and only if a $f(\phi)$ which we construct in M, decides M-SAT, to achieve this, let M_{SAT} be a TM that decides M-SAT

Let M = "On input $\langle \phi \rangle$:

1. let $k = m$
2. construct $f(\phi) = \langle \phi, k \rangle$
3. Run M_{SAT} on input $f(\phi)$, if M_{SAT} accepts, accept, if M_{SAT} rejects, reject

Since M-SAT accepts only if $k = \max(\phi)$, we construct $f(x)$ such that k becomes the max number of conjuncts in ϕ so suppose $\phi \in$ SAT, that means there is a satisfying assignments, so the number of conjuncts that can be made true is equal to the maximum amount of conjunct, thus $f(x)$ is in M-SAT, the same applies backwards, since $f(x)$ is in M-SAT, we then know that k is the same as the max amount of conjuncts, so every conjunct can be made true, so ϕ is in SAT, for time the main time it takes is to find the maximum amount of conjuncts in ϕ so if $|\phi| = n$, to iterate and find the maximum count of conjuncts takes n time, thus this is polynomial.

Problem 3: Space Complexity (30 points)

1. (30 points) We define the following language:

$$EQ_{re} = \{\langle e_1, e_2\rangle | e_1, e_2 \text{ are regular expressions and } L(e_1) = L(e_2)\}$$

Show that $EQ_{re} \in$ PSPACE.

Solution:
We construct a non-deterministic Turing Machine M for $\overline{EQ_{re}}$.
Let M ="On input $\langle e_1, e_2\rangle$:

1. Use Lemma 1.55 from Sipser's book to convert e_1 and e_2 into two NFA, N_1 and N_2.
2. Let $S_1 = \{\text{start state of } N_1\}$ and $S_2 = \{\text{start state of } N_2\}$. Let $C = 0$.
3. If exactly one of S_1, S_2 contains an accept state, accept.
4. If $C \geq n - 1$, where n is the number of states in the NFA with the most amount of states.
5. non-deterministically guess a symbol a from their alphabet Σ.
6. Replace S_1 with the set of states reachable from S_1 on a, and do the same for S_2"

Now we construct a Turing machine M' for EQ_{re}.
Let M' ="On input $\langle e_1, e_2\rangle$:

1. Run M on input $\langle e_1, e_2\rangle$.
2. If M accepts, reject.
3. If M rejects, accept.

Correctness:

Since M only accepts when exactly one of the NFAs reaches an accepting state for some w, we can conclude that w is only in one of the two NFAs, making them different. And since M' only accepts when M rejects, M accepts only when e_1 and e_2 are equal.

Space complexity analysis:

Each S_i is a set of states, we only store the states we are currently in, which can be at most n at a time, so $O(n)$.
The counter can only reach a maximum of $n - 1$, so we only need $\lfloor \log_2(n) \rfloor + 1$ to represent it. So $O(\log_2(n))$
The RegExp to NFA construction is also $o(n)$ space complexity.
So the full space complexity is $O(n) + O(\log_2(n)) + O(n) = O(n)$